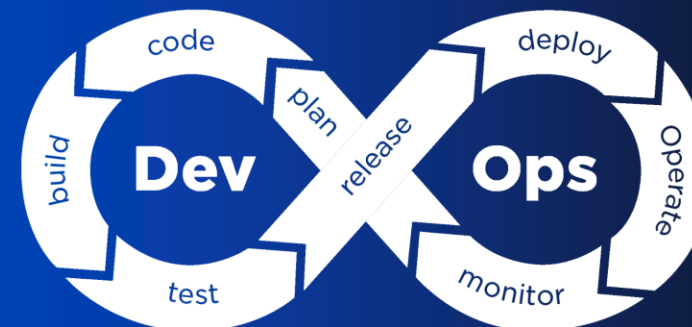


Containerização I

Semana 2

CSDV-246 - DevOps



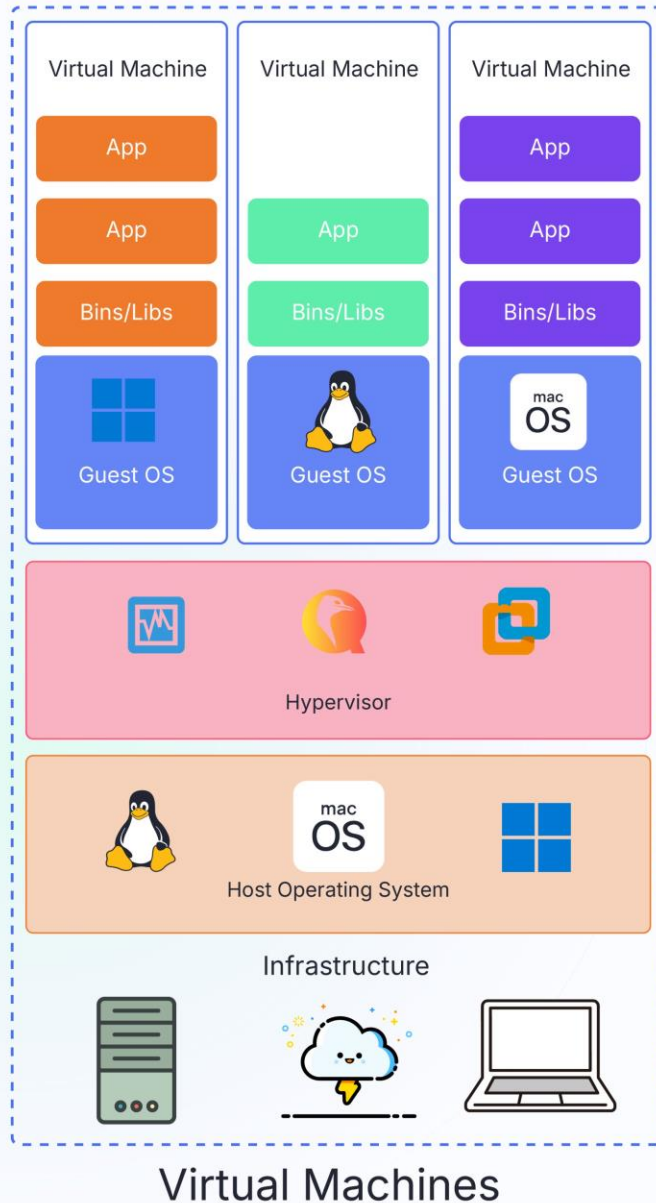
Conteúdo

- Virtualização x Containerização
- Definição de contêiner
- A Iniciativa de Contêiner Aberto (OCI)
- Docker e sua arquitetura
- Ciclo de vida do contêiner
- Gerenciamento de contêineres
- Persistência de dados
- Redes
- Construção de imagem
- *Registries*

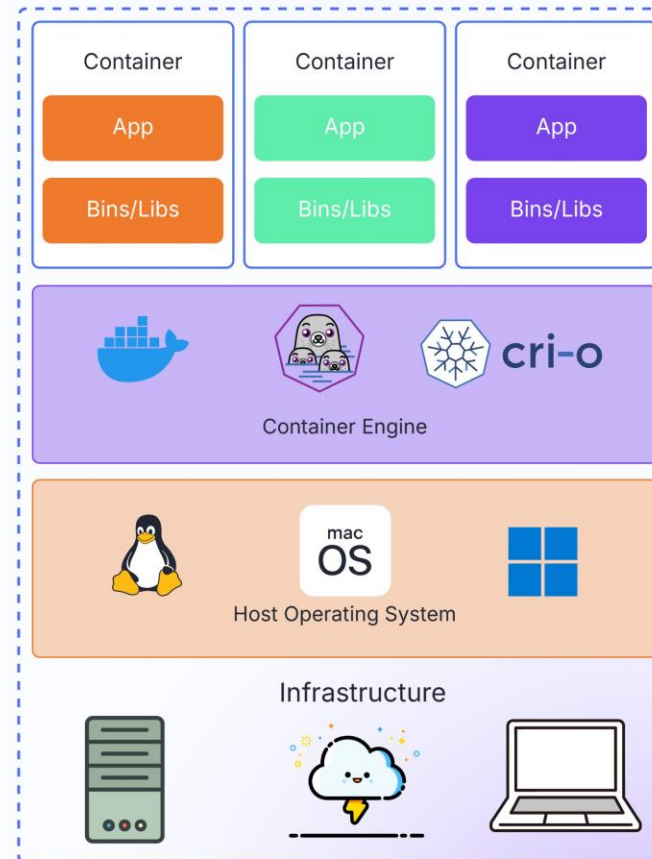


Virtualização x Containerização

VMs vs CONTAINERS



Containers



Máquinas Virtuais

- **Peso:** Geralmente pesado, com inicialização lenta e arquivos repetidos se comparado ao sistema operacional host.
- **Custo de manutenção:** Semelhante a uma máquina real.
- **Formatos:** VDI, VMDK, VHD, raw, etc.

Contêineres

- **Peso:** Eles tendem a ser mais leves que MVs, sem inicialização e kernel.
- **Custo de Manutenção:** Portátil e escalável.
- **Formatos:** Padrão dentro do OCI.

Definição de Contêiner

Definição de Contêiner



Antes



Agora



Definição de Contêiner



- Um contêiner armazenará um aplicativo ou software em um sistema de arquivos isolado.
- Isso significa que ele contém tudo o que é necessário para ser executado: código, *runtimes*, ferramentas do sistema, bibliotecas, qualquer coisa que possa ser instalada em um servidor.
- Isso garante que o software sempre será executado da mesma forma, independentemente do seu ambiente.

Definição de Contêiner



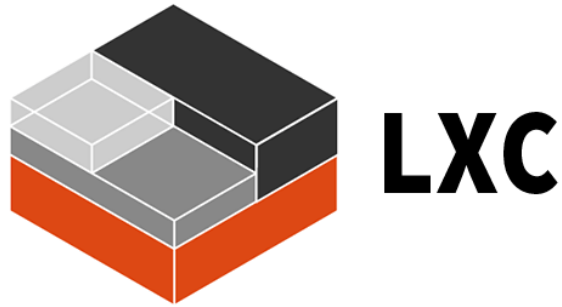
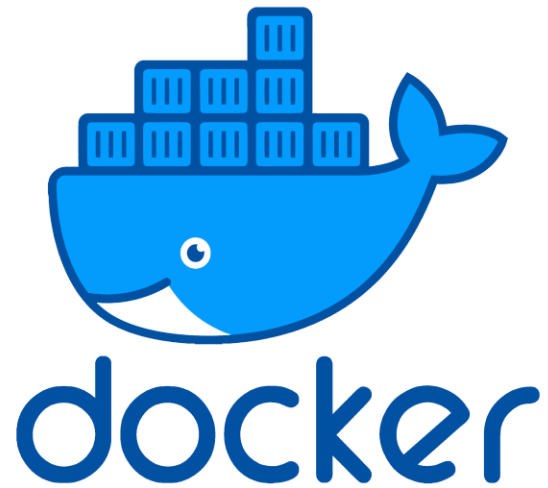
- Um contêiner é uma instância em execução.
- Podemos interagir com eles e mudar seu status.
- Eles tendem a ser leves.
- Com baixo acoplamento.
- Eles são escaláveis.
- Seguros.
- Eles são unidades lógicas.

Definição de Container



- A finalidade dos containers é obter um ambiente isolado do sistema operacional do host.
- Executar processos em um ambiente isolado com segurança.
- O host Linux e os containers compartilham o sistema de arquivos do kernel.
- Os containers são executados como um processo comum no Linux.

Algumas Ferramentas



podman

Open Container Initiative (OCI)

Open Container Initiative (OCI)



- Um projeto de software livre que define padrões comuns para criar e executar contêineres.
- Seu objetivo é garantir que os contêineres sejam compatíveis com diferentes ferramentas e plataformas, independentemente de quem os desenvolve ou executa.
- O Docker é um dos membros fundadores desta iniciativa desde 2015.

Open Container Initiative (OCI)



- Visite o site oficial para obter mais informações:
<https://opencontainers.org>

Our Members



Docker e sua Arquitetura

A Docker Inc.



- A Docker Inc. é uma empresa com sede em São Francisco, Vale do Silício.
- Foi inicialmente chamado de "dotCloud".
- Eles tinham uma ferramenta interna para gerenciar contêineres chamada "Docker".
- Eles aproveitaram a tecnologia de "Linux Containers".
- Em 2013, o dotCloud foi renomeado como Docker Inc.

A Docker Inc.



- A Docker Inc. fornece uma plataforma com diferentes produtos, como:
 - Docker Engine
 - Docker Desktop
 - Docker Build
 - Docker Compose
 - Docker Hub
 - Docker Swarm
 - e outros...

Docker Engine



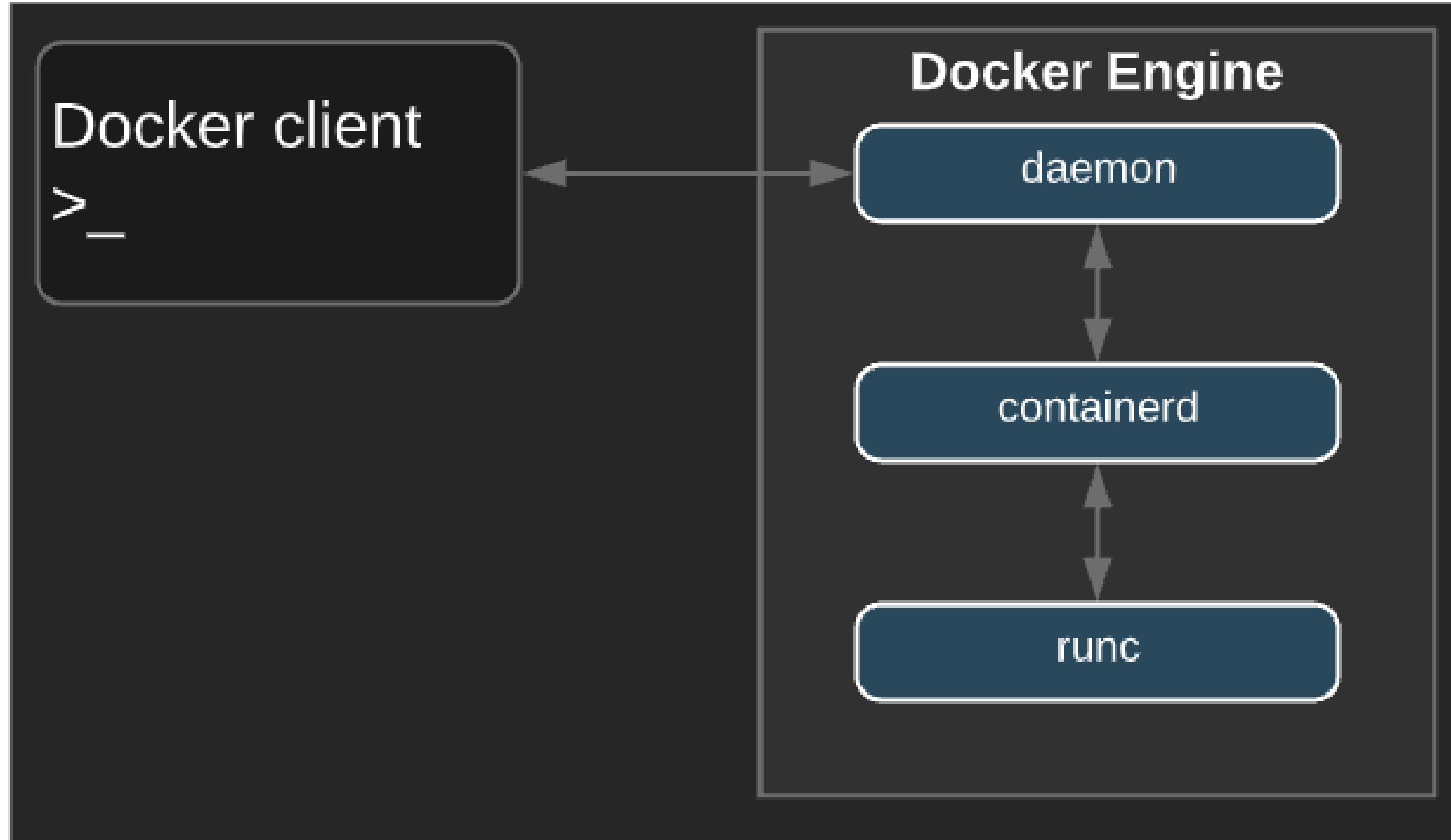
- Agora vamos falar sobre o Docker Engine.
- É uma tecnologia de código aberto para aplicativos de containerização.
- Ele foi projetado para ajudar na implantação de aplicativos em diferentes ambientes.
- Desenvolvido em linguagem de programação Go.
- Ele usa a tecnologia de "*namespaces*" para fornecer um espaço de trabalho isolado chamado contêiner.
- Funciona como um aplicativo cliente-servidor.

Docker Engine



- Possui os seguintes componentes:
 - Docker Client (Docker CLI)
 - Docker API
 - Docker Daemon (dockerd)
 - Containerd
 - RunC
- Esses componentes trabalham juntos para criar e levantar contêineres.

Docker Engine



Docker Engine



Docker Client (Docker CLI)

- É a interface de linha de comando (CLI) que os usuários usam para interagir com o Docker *Daemon*.
- Ele permite que você gerencie contêineres, imagens, redes e volumes, por meio de comandos que são inseridos em um terminal.
- Usa a API do Docker para se comunicar com o *Daemon* do Docker.

Docker Engine



Docker Daemon (dockerd)

- É o processo que escuta as solicitações da API do Docker e gerencia os "Objetos do Docker", como imagens, contêineres, redes e volumes.
- Ele atua como o "cérebro" do Docker Engine.
- Ele recebe as ações solicitadas pelo Docker Client, processa-as e as encaminha para a API do Docker, onde operações como criar, excluir, etc. são executadas.

Docker Engine



containerd

- É um *runtime* de contêiner padrão da indústria.
- Gerencia o ciclo de vida do contêiner.
- Ele foi projetado para se integrar a sistemas maiores (como Docker ou Kubernetes), em vez de ser usado diretamente por desenvolvedores ou usuários finais.

Docker Engine



runc

- É um *runtime* de contêiner leve e portátil e visa criar contêineres padronizados.
- O Docker o usa para criar e executar contêineres no Linux de acordo com as especificações do OCI.
- Não depende da plataforma Docker.

Docker Engine



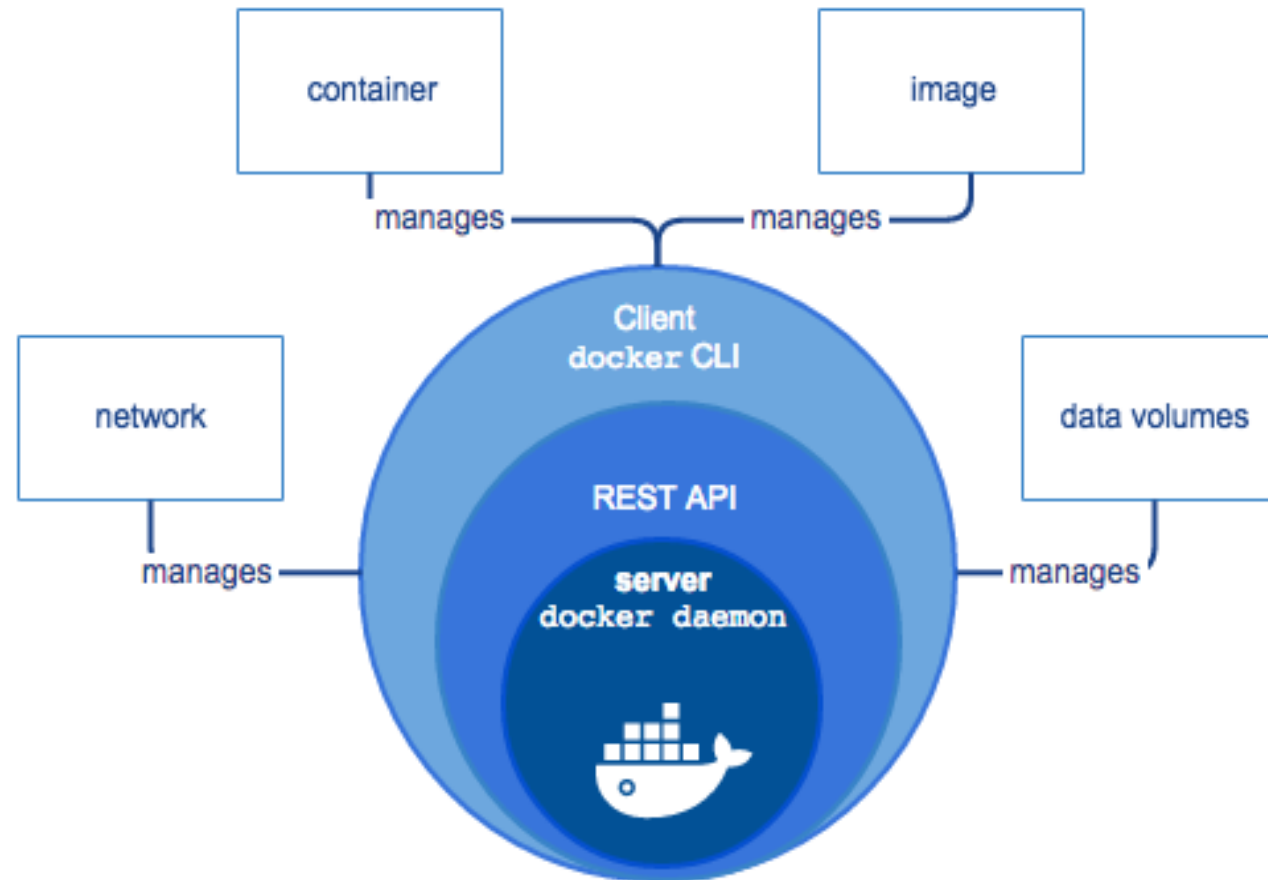
Objetos do Docker

- Imagens
- Contêineres
- Redes
- Volumes
- Plugins
- Outros...

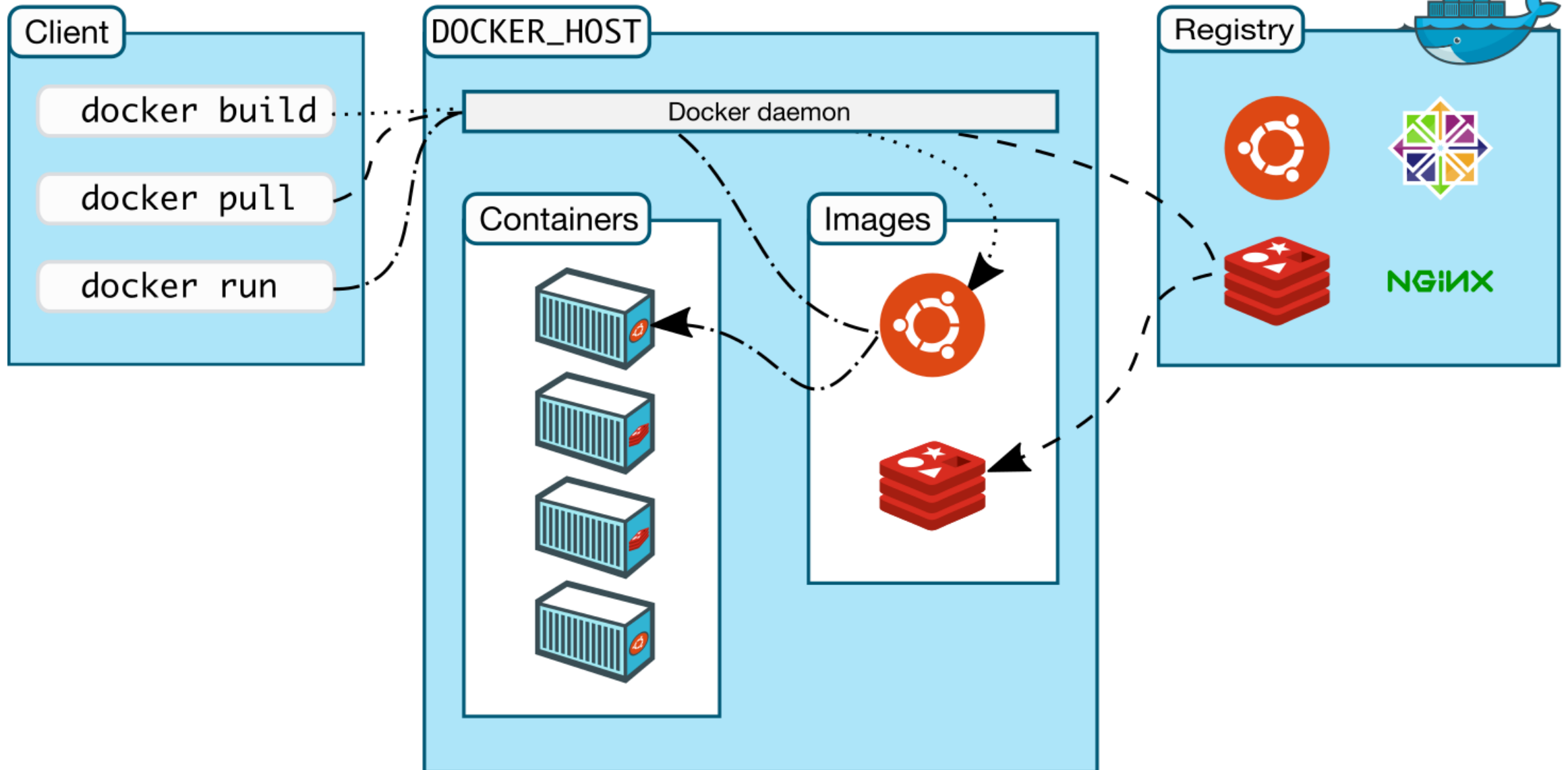
Docker Engine



Objetos do Docker



Arquitetura do Docker



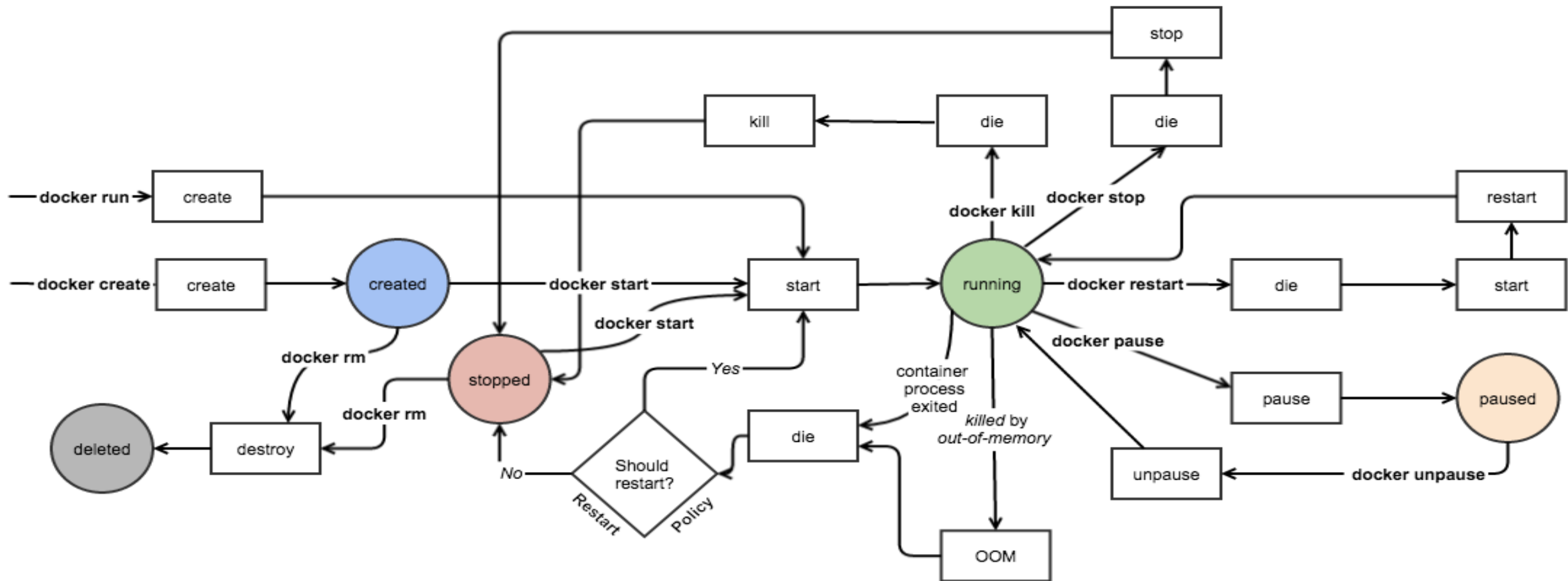
Ciclo de vida do Contêiner

Ciclo de vida do Contêiner



- Esses são os estágios pelos quais um contêiner passa desde o momento em que é criado até ser excluído.
- Esses estágios refletem as operações executadas no contêiner e como o Docker o gerencia em cada fase.

Ciclo de vida do Contêiner



Ciclo de vida do Container



Criar

- Comando:
`docker container create --options [IMAGEM]`
- Exemplo:
`docker container create --name mongo mongo:latest`

Ciclo de vida do Container



Iniciar

- Comando:
`docker container start [NOME OU ID DO CONTÊINER]`
- Exemplo:
`docker container start mongo`
`docker container start 32be87`

Ciclo de Vida del Contenedor



Criar e iniciar

- Comando:
`docker container run --options [IMAGE]`
- Exemplo:
`docker container run --name mongo -d --restart always mongo:latest`
- Lista completa de opções [aqui](#)

Ciclo de Vida del Contenedor



Parar

- Comando:
`docker container stop [NOME OU ID DO CONTÊINER]`
- Exemplo:
`docker container stop mongo`
`docker container stop 32be87`

Ciclo de Vida del Contenedor



Excluir

- Comando:
`docker container rm [NOME OU ID DO CONTÊINER]`
- Exemplo:
`docker container rm mongo`
`docker container rm 32be87`

Gerenciamento de Contêineres

Gerenciamento de Containers



- É o processo de gerenciar, organizar e otimizar o ciclo de vida dos containers.
- Isso envolve o gerenciamento eficiente de aplicativos em containers, garantindo que eles estejam disponíveis, escaláveis, seguros e operando de maneira ideal em ambientes de desenvolvimento e produção.

Gerenciamento de Containers



- Pontos-chave do gerenciamento de containers:
 - Criação e implantação
 - Monitoramento e manutenção
 - Persistência de dados
 - Redes
 - Gerenciamento de imagens
 - Orquestração

Gerenciamento de Containers



Criação e implantação

- Gerenciar a criação de containers a partir de imagens e sua implantação em ambientes de desenvolvimento, teste ou produção.
- Não é recomendado criar containers a partir de imagens com a tag "latest", você deve usar tags específicas, como "7.0", "7.0-jammy".
- Já vimos o Ciclo de Vida do Container, onde foi explicada a criação e remoção de containers usando o Docker.

Gerenciamento de Containers



Criação e implantação

- Listar containers no status "running":
docker container list
- Outras maneiras de listar containers:
docker container ls
docker ps
- Listar containers no status "running" y "exited":
docker container list --all
docker container ls -a
docker ps -a

Gerenciamento de Containers



Monitoramento e Manutenção

- Refere-se ao monitoramento do desempenho dos containers (CPU, memória, uso de rede, etc.) e mantê-los íntegros, reiniciar ou substituir containers com falha automática ou manualmente.
- Debugging:
 - Verificar logs
 - Acessar o container
 - Monitorar estatísticas de consumo de recursos

Gerenciamento de contêineres



Monitoramento e Manutenção

- Inspecionando um contêiner:
docker container **inspect** [NOME OU ID DO CONTÊINER]
- Exemplo:
docker container **inspect** mongo
docker **inspect** 32be87

Gerenciamento de Containers



Monitoramento e Manutenção

- Monitorar logs:
`docker container logs -f [NOME OU ID DO CONTÊINER]`
- Exemplo:
`docker container logs -f mongo`
`docker logs -f 32be87`

Gerenciamento de Containers



Monitoramento e Manutenção

- Acessar um container:
`docker container exec -it [NOME OU ID DO CONTÊINER]
[command]`
- Exemplo:
`docker container exec -it mongo bash`
`docker exec -it 32be87 bash`

Gerenciamento de Containers



Monitoramento e Manutenção

- Monitorar estatísticas de consumo de recursos:
docker container **stats -a**
- Exemplo:
docker container **stats -a**
docker **stats -a**

Gerenciamento de contêineres



Persistência de dados

- Por padrão, as informações geradas dentro dos contêineres são efêmeras, ou seja, quando o contêiner é excluído, todos os dados adjacentes também serão excluídos.
- Os contêineres têm a capacidade de manter os dados armazenados por um aplicativo, mesmo quando o contêiner que o executa é interrompido, excluído ou recriado.
- Vamos nos aprofundar nisso na seção **Persistência de dados**, mais tarde.

Gerenciamento de contêineres



Redes

- São mecanismos que permitem que um contêiner se comunique com outros contêineres (e vice-versa) e com outras redes externas.
- Isso é fundamental para que os aplicativos distribuídos em contêineres funcionem corretamente e compartilhem informações.
- Entraremos em mais detalhes sobre isso na seção **Networking**, mais tarde.

Gerenciamento de Containers



Gerenciamento de imagens

- É o processo de gerenciamento das imagens que são usadas para criar containers.
- Abrange a criação, armazenamento, distribuição, atualização e otimização de imagens.
- Uma **imagem de container** é um modelo imutável que contém tudo o que é necessário para executar um aplicativo, incluindo código, bibliotecas, dependências e configurações.

Gerenciamento de Containers



Gerenciamento de Imagens

- Baixar imagens, por padrão, do Docker Hub

```
docker image pull [NOME DA IMAGEM]:[TAG]
```

- Exemplo:

```
docker image pull mongo:latest
```

```
docker image pull mongo:7.0
```

- Exemplo: Baixar de outro repositório

```
docker image pull gcr.io/google-samples/hello-app:1.0
```

Gerenciamento de Containers



Gerenciamento de imagens

- Carregar imagens, por padrão, para um repositório no Docker Hub:

```
docker image push [REPOSITARIO]/NOME DA  
IMAGEM]:[TAG]
```

- Exemplo:

```
docker image push myrepo/mongo:7.0
```

Gerenciamento de Containers



Gerenciamento de imagens

- Excluir:
`docker image rm [NOME DA IMAGEM]:[TAG]`
- Exemplo:
`docker image rm mongo:7.0`
`docker rmi mongo:7.0`

Gerenciamento de Contêineres



Gerenciamento de imagens

- Listar Imagens:
docker image list
- Outras maneiras de listar imagens:
docker image ls
docker images

Gerenciamento de Containers



Gerenciamento de imagens

- Marcar imagens:
`docker image tag [NOME DA IMAGEM]:[TAG] [NOVO NOME]:[NOVA TAG]`
- Exemplo:
`docker image tag mongo:7.0 mymongo:mytag-7.0`
- Criação de imagem:
Vamos estudar a criação de imagens na seção "**Construção de imagens**"

Gerenciamento de Containers



Orquestração

- À medida que os aplicativos modernos se dividem em vários serviços executados em containers, você precisa gerenciar com eficiência esses containers e como eles interagem entre si, especialmente quando distribuídos em vários servidores ou em um cluster.
- Esse processo automatiza a implantação, o gerenciamento, o dimensionamento e a orquestração de containers em escala.

Persistência de dados

Persistência de dados



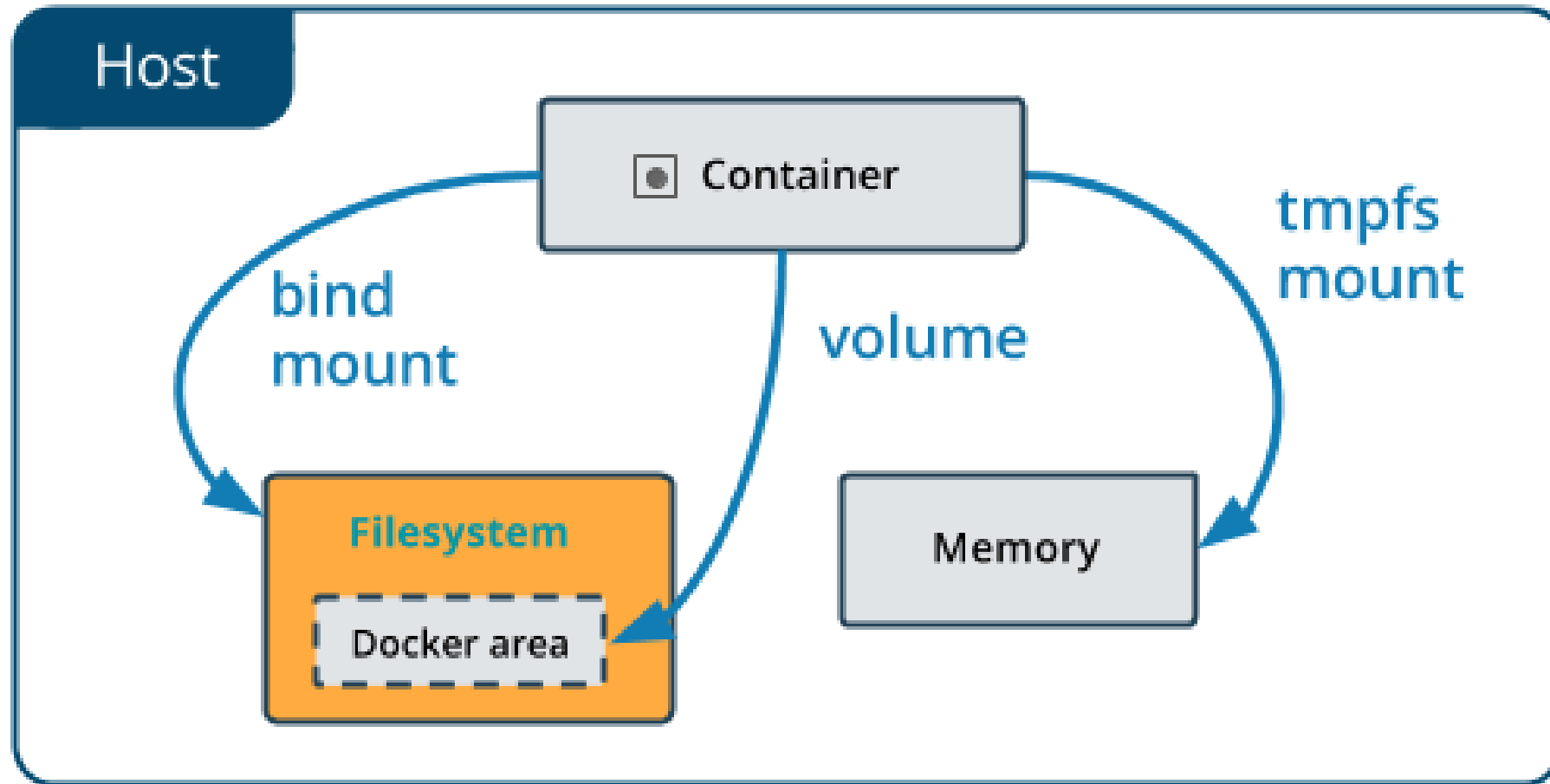
- Como vimos anteriormente, os contêineres são efêmeros por padrão.
- Muitas vezes, vamos querer que as informações geradas dentro dos containers sejam "salvas" fora, como bancos de dados, arquivos gerados por aplicativos, configurações, etc.
- Vamos aproveitar os mecanismos de persistência para compartilhar dados entre contêineres e o sistema de arquivos HOST (ou vice-versa).

Persistência de dados



- Isso nos permite fazer backup e restaurar as informações obtidas dos contêineres.
- Se um contêiner for excluído, outro poderá ser recriado para reutilizar os dados persistentes.
- Docker fornece os seguintes mecanismos:
 - Bind Mounts
 - Volumes
 - Tmpfs Mounts

Persistência de dados



Persistencia de Datos



Bind Mounts

- Permite que o contêiner acesse diretamente um diretório no sistema de arquivos do host.
- Basicamente, você cria um diretório no sistema operacional host e a "vincula" a um contêiner.
- O Docker não gerencia ou otimiza esses diretórios, a administração delas recai sobre o usuário host.

Persistencia de Dados



Bind Mounts

- Considerações:
 - **Segurança e Permissões:** Pode ser um gateway para um invasor obter privilégios no sistema host.
 - **Integridade das informações:** Qualquer usuário ou grupo de hosts com privilégios suficientes pode alterar as informações.
- Casos de Uso:
 - Compartilhamento de código-fonte.
 - Acesso de arquivos e logs de configuração.
 - Compartilhar dados entre diferentes contêineres e o host.

Persistencia de Datos



Bind Mounts

- Criar um diretório no sistema operacional host
`mkdir ~/mongo_data`
- Criar um contêiner com Bind Mount usando o comando-v:
`docker run -d --name mongo -v
~/mongo_data:/data/db mongo:7.0`

Persistência de dados



Volumes

- Mais recomendado
- São diretórios **gerenciados pelo Docker**, ou seja, podemos gerenciá-los por meio de comandos do Docker.
- Eles são mais seguros, portáteis e fáceis de administrar do que os *bind mounts*.
- Basicamente, quando um volume é usado, um novo diretório é criado dentro do "Diretório de Armazenamento do Docker" no sistema operacional host e o Docker gerencia o conteúdo desse diretório.

Persistência de dados



Volumes

- Amplamente utilizado em ambientes de produção para bancos de dados, aplicativos da Web e sistemas distribuídos.
- Considerações:
 - **Integridade das informações:** Alterações manuais no volume não são recomendadas.
 - **Volumes órfãos:** Quando você exclui contêineres sem limpar os volumes associados, eles se acumulam e consomem espaço no disco rígido.
 - **Exclusão acidental:** Informações importantes podem ser perdidas porque os dados armazenados em um volume não podem ser recuperados após a exclusão. O backup é recomendado.

Persistência de dados



Volumes

- Crie um volume:
`docker volume create [NOME DO VOLUME]`
- Exemplo:
`docker volume create mongo_data`
- Listar todos os volumes locais:
`docker volume ls`
- Inspecionar um volume:
`docker volume inspect mongo_data`

Persistência de dados



Volumes

- Crie um contêiner com um volume:

```
docker run -d --name mongo -v mongo_data:/data/db mongo:7.0
```

Persistência de dados



Tmpfs Mount:

- Permite que um sistema de arquivos temporário seja montado diretamente na RAM do host e não no disco rígido.
- Acesso rápido aos dados,
- Quando o contêiner é interrompido, reiniciado ou excluído, os dados são excluídos imediatamente.
- Seu uso adequado pode melhorar significativamente o desempenho em cenários específicos, mas seu uso deve ser equilibrado considerando a quantidade de memória disponível no sistema.

Persistência de dados



Tmpfs Mount:

- Permite que um sistema de arquivos temporário seja montado diretamente na RAM do host
- Casos de uso:
 - **Arquivos temporários:** Caches, logs temporários, dados confidenciais.
 - **Melhore o desempenho:** Reduza a E/S no disco rígido.
- Exemplo

```
docker run -d --name mongo --tmpfs /data/db:rw,noexec,  
nosuid,size=50M mongo:7.0
```

Redes



- Já vimos que as redes de contêineres permitem que os contêineres se conectem uns aos outros, ao host e a redes externas.
- O Docker oferece diferentes tipos de redes:
 - Bridge
 - Host
 - None

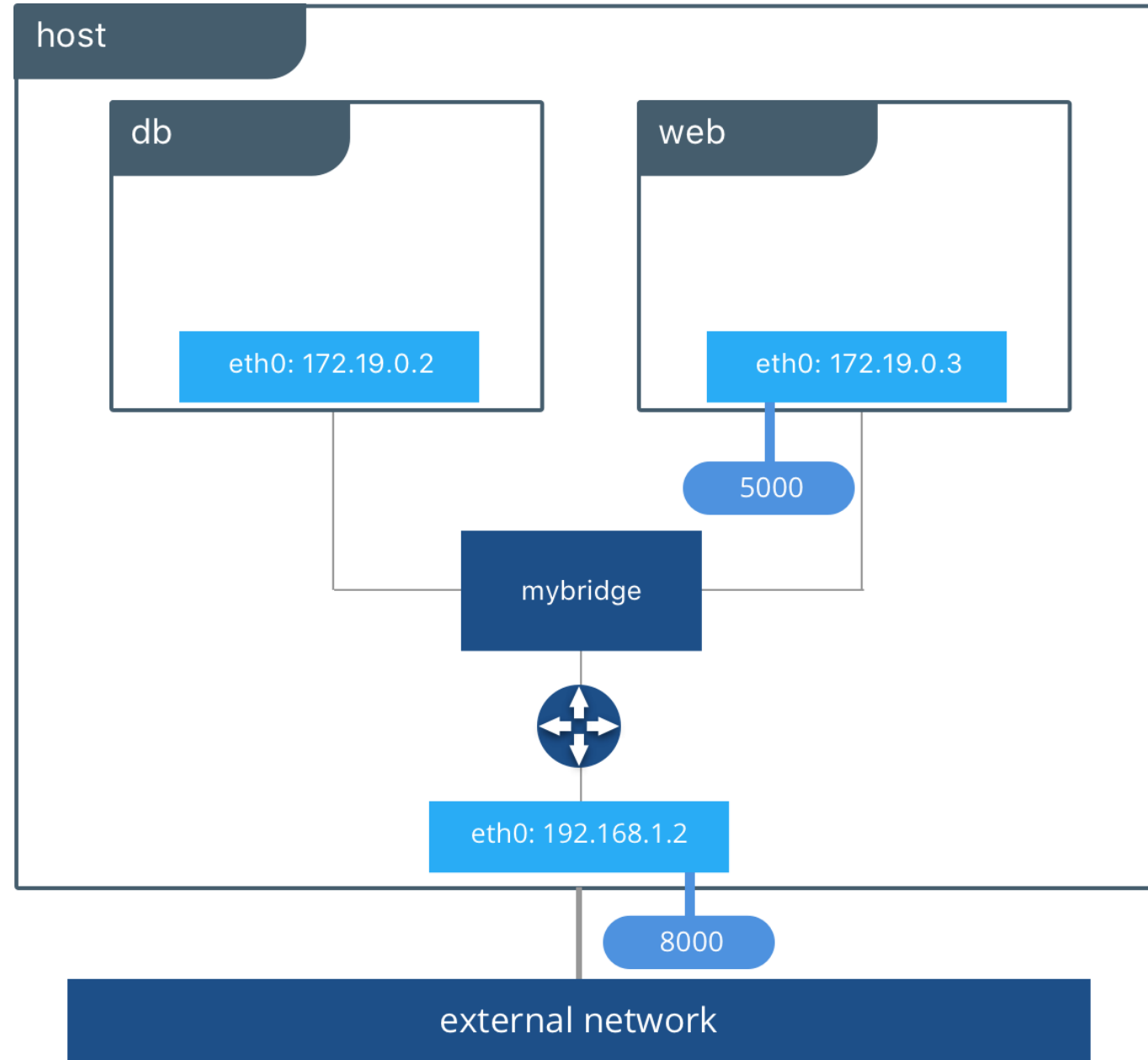


Bridge:

- Tipo de rede padrão atribuído quando um contêiner é criado
- Os contêineres conectados a essa rede precisam ter portas expostas configuradas para serem acessíveis de fora do host.
- Cada contêiner receberá um endereço IP.
- **IMPORTANTE:** Os contêineres podem se comunicar entre si, mas apenas por **seus endereços IP**. Recomenda-se trabalhar com redes interligadas do tipo **"User-defined bridge"**.

Redes

Bridge:





User-defined bridge

- Essas são redes do tipo *bridge* que os usuários podem criar e gerenciar manualmente.
- Eles têm maiores vantagens do que a rede Bridge padrão:
 - **Isolamento entre contêineres.**
Os contêineres podem ser isolados de outras redes.
 - **Resolução automática de nomes DNS.**
Os contêineres podem se comunicar entre si usando seus nomes em vez de endereços IP.
 - **Flexibilidade na configuração.**
Um contêiner pode ser conectado a diferentes redes *bridge*.

Redes



User-defined bridge

- Criar uma nova rede do tipo "User-defined bridge"
docker network create [NOME DA REDE]
- Exemplo:
docker network create mongonet
- Listar todas as redes
docker network ls

Redes



User-defined bridge

- Inspecionar uma rede
docker network **inspect** [NOME DA REDE]
- Conectar um contêiner a uma rede existente:
docker network **connect** [NOME DA REDE] [NOME DO CONTÊINER]
- Desconectar um contêiner a uma rede
docker network **disconnect** [NOME DA REDE] [NOME DO CONTÊINER]



User-defined bridge

- Criar contêiner e conectar-se a uma rede "User-defined bridge" ao mesmo tempo.

```
docker run -d --name mongo --network [NOME DA REDE] mongo:7.0
```



Host

- Use a rede do sistema operacional do host diretamente.
- O contêiner não obtém seu próprio endereço IP, mas "compartilha" o endereço IP e as portas do sistema operacional host. Portanto, não há necessidade de publicar portas para disponibilizar o aplicativo de redes externas.
- A rede host está disponível apenas em sistemas Linux, **não funciona no Windows**.



Host

- Crie um contêiner e conecte-se a uma rede host ao mesmo tempo.

```
docker run -d --name mongo --network host mongo:7.0
```



None

- O contêiner não terá acesso à rede externa nem poderá se comunicar com outros contêineres.
- Crie contêineres isolados que não precisem de acesso à rede, como aplicativos que executam apenas processamento local ou testes internos.
- Exemplo:
`docker run -d --name mongo --network none mongo:7.0`

Construção de Imagens

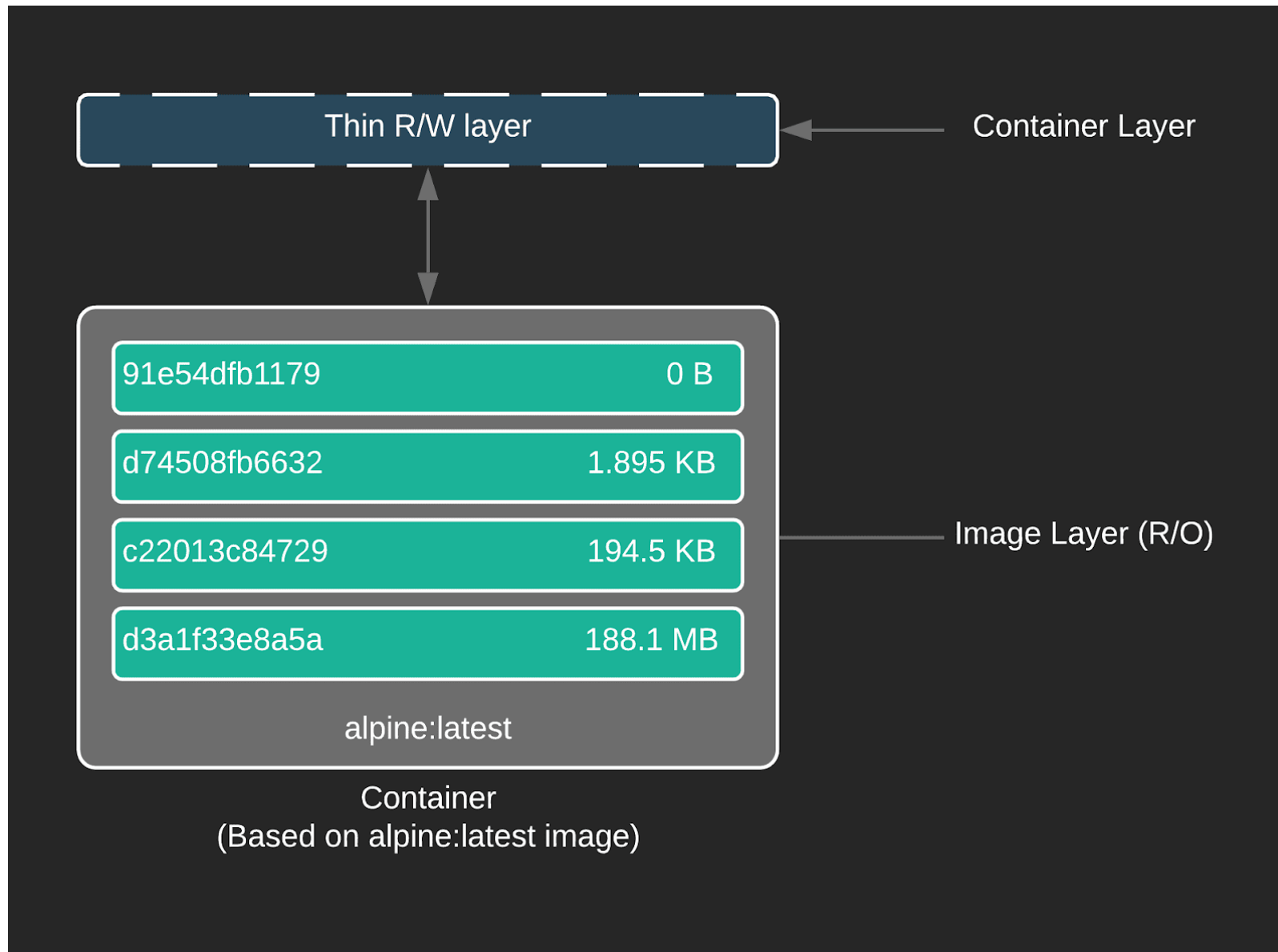
Construção de Imagens



O que é uma imagem de contêiner?

- Uma imagem de contêiner é um pacote padrão que inclui todos os arquivos, binários, bibliotecas e configurações para executar um contêiner.
- Eles são a base para a criação de contêineres, e cada contêiner pode ser replicado em qualquer ambiente.
- Existem 2 princípios-chave da imagem de contêiner:
 - As imagens são **Imutáveis**.
 - Eles são compostos de **Camadas**.

Construção de Imagem



Construção de Imagens



Dockerfile

- Existem várias ferramentas que permitem criar imagens de contêiner, por exemplo, Buildpacks.io, Podman, Docker Build, etc.
- O Docker pode criar imagens automaticamente lendo uma série de instruções declaradas em um Dockerfile.
- Um Dockerfile é um documento de texto que contém todos os comandos que um usuário pode chamar na linha de comando para montar uma imagem.

Construção de Imagens



Instruções mais comuns

Instrução	Descrição
FROM <image>	Definir uma imagem base sobre o qual uma nova imagem será construída.
RUN <command>	Execute qualquer comando em uma nova camada acima da imagem atual e confirma o resultado. Ele também possui um shell para executar comandos.
WORKDIR <directory>	Define o diretório, a partir do qual, qualquer comando nas instruções será executado RUN, CMD, ENTRYPOINT, COPY, e ADD
COPY <src> <dest>	Copie novos arquivos e pastas (que estão dentro do Build Context) de um diretório de origem e os adiciona ao sistema de arquivos do contêiner em um caminho de destino.
CMD <command>	Ele permite que você defina o programa padrão que será executado assim que o novo contêiner for inicializado com base nessa nova imagem. Cada Dockerfile tem apenas um CMD.

Construção de Imagens



```
# syntax=docker/dockerfile:1
```

```
FROM ubuntu:22.04
```

```
# install app dependencies
```

```
RUN apt-get update && apt-get install -y python3 python3-pip
```

```
RUN pip install flask==3.0.*
```

```
# install app
```

```
COPY hello.py /
```

```
# final configuration
```

```
ENV FLASK_APP=hello
```

```
EXPOSE 8000
```

```
CMD ["flask", "run", "--host", "0.0.0.0", "--port", "8000"]
```

Construção de Imagens



Build Context

- Este é o conjunto de arquivos que podem ser acessados pelo Docker Build.
- Quando o Contexto de Compilação é um diretório local, ele se torna o conjunto de arquivos que o construtor pode acessar durante o processo de criação da imagem.
- As instruções do Dockerfile, como COPY e ADD, podem se referir a qualquer um dos arquivos, diretórios e subdiretórios no contexto.

Construção de Imagens



Build Context

- Recomenda-se posicionar o Dockerfile na pasta raiz do projeto de código.
- O caminho do Contexto de Compilação é passado como um argumento para o comando "docker build".
- É muito comum definir o Contexto de Compilação para a pasta atual (.) onde o Dockerfile está localizado.

docker build .

Construção de Imagens



Comando "docker build"

- Criar imagem:
docker build -f [NOME DO DOCKERFILE] [CONTEXTO]
- Se o arquivo Dockerfile for chamado de Dockerfile por padrão:
docker build .
- Se o Dockerfile tiver um nome e um caminho diferentes:
docker build -f folder/myDockerfile folder/.
- Marcar a nova imagem quando o edifício estiver concluído:
docker build -t myimage:v1.0 .

Construção de Imagens



Multi-stage Builds

- Eles são usados para otimizar Dockefile, ou seja, para torná-los fáceis de ler e manter.
- Várias instruções FROM são usadas.
- Cada instrução FROM pode usar uma imagem base diferente.
- Cada FROM inicializa um novo estágio da compilação.
- Os arquivos podem ser copiados de um estágio para outro, evitando copiar arquivos que não são necessários para a imagem final resultante.

Construção de Imagens



```
# syntax=docker/dockerfile:1
FROM golang:1.23 AS build
WORKDIR /src
COPY src/main.go .
RUN go build -o /bin/hello ./main.go

FROM scratch
COPY --from=build /bin/hello /bin/hello
CMD ["/bin/hello"]
```

Registries

Registries



O que é um Registry?

- É um repositório, ou conjunto de repositórios, usado para armazenar imagens de contêiner.
- Permite , compartilhar, distribuir e controlar a versão de imagens.
- Ele também permite que você controle o acesso e as permissões.
- Além disso, eles podem realizar análises de vulnerabilidade das imagens.

Registries



Registries



Comando "docker login"

- Login to Docker Hub:
`docker login -u dockerhub_username`
- Login to GitLab Registry:
`docker build -u Gitlab_username registry.gitlab.com`
- Login to a Private Registry:
`docker build -u username private.registry.url`
- Storing credentials:
`$HOME/.docker/config.json`

Registries



Marcar e fazer upload de imagens para um Registry

- marcar a tag com o domínio do registry:
`docker tag imagen:1.0 registry.gitlab.com/repo/nombre_imagen:v1.0`
- Carregar imagem para o Registro (necessário fazer login primeiro):
`docker push registry.gitlab.com/repo/nombre_imagen:v1.0`

Q&A